BEAVER'S CHOICE PAPER COMPANY

Multi-Agent Quote, Inventory and Sales System

Comprehensive Project Report

Architecture, Agents, Testing, Live Runs and Results

Framework: smolagents ToolCallingAgents

Five agents, twelve tools, seven helper functions, one SQLite database

Evaluation: 20 sample requests, offline run and three live runs

October 7, 2026

Contents

1. Executive Summary
2. Problem Statement
3. Solution Statement
4. Dataset
5. Architecture
6. Multi-Agents
6.1 ToolCallingAgents
6.2 inventory_agent
6.3 quote_agent
6.4 finance_agent
6.5 sales_agent
6.6 Orchestration tools
6.7 Helper functions
6.8 Database
7. Testing
8. Results
9. Value Statement
10. Limitations
11. Conclusion
12. References

1. Executive Summary

This report describes a five-agent system built for the Beaver's Choice Paper Company to answer customer quote requests, check inventory, protect cash, and record completed sales. The company receives free-text inquiries that mix product wording, quantities, and delivery deadlines. Each inquiry must be turned into an accurate, explainable answer that respects stock levels, supplier lead times, and the cash position of the business.

The solution uses the smolagents framework. One orchestrator agent parses each inquiry and delegates to four worker agents: inventory, quote, finance, and sales. Business rules live in plain Python functions that the agents call as tools, and the agents share a request-scoped order ledger so that they pass an order key instead of retyping data. Seven helper functions supplied with the project provide all database access.

The system was evaluated on the 20 requests in the sample file. Ten orders were fulfilled in full, nine were fulfilled in part, and one was rejected in full of a clear reason. Of 60 distinct product lines requested, 42 were supplied and 18 were declined with a stated reason: 14 because the supplier could not deliver by the deadline and 4 because the product is not in the catalog. Cash rose from $45,059.70 to $46,804.87, and total assets rose from $50,000.00 to $51,209.42.

The system matters because the cost of a wrong answer is real. A confirmed order that cannot be delivered damages a customer relationship, and a booked sale that is written twice distorts the financial record. The design therefore treats accuracy and honesty as first-class goals: the agents explain every refusal, and the code guarantees that each order is booked at most once.

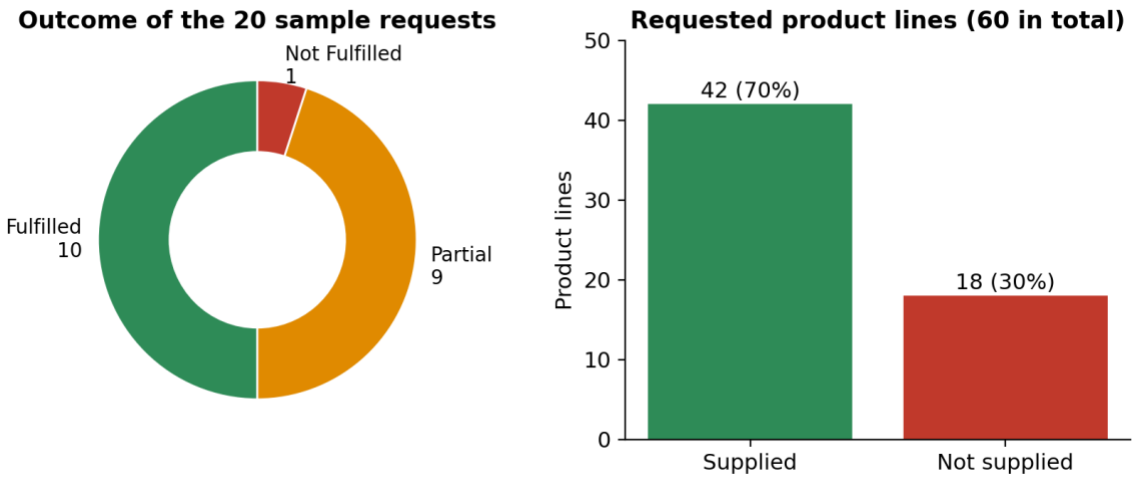


Figure 1: Outcome of the 20 sample requests and the product lines behind them.

The system was also run three times against a live language model. The first run exposed serious weaknesses: invented order keys, a hallucinated zero cash balance, replies that leaked internal terms, and replies that contradicted the database. The second run produced correct replies but revealed a duplicate booking defect that overstated cash by $290.43 across three orders. After each finding was fixed and reproduced under test, the third live run matched the offline run exactly, including the final cash balance of $46,804.87.

Table 1: Headline results

Measure	Result
Requests evaluated	20
Fulfilled in full / in part / rejected	10 / 9 / 1
Product lines supplied / declined	42 / 18
Sales revenue booked	$4,142.92
Final cash balance	$46,804.87
Built-in requirement checks passed	7 of 7
Live run 3 matches offline run	Yes, every request

The report that follows covers the problem, the solution, the data, the architecture, each agent, the test strategy, the results, and the value delivered. It closes with limitations, a conclusion, and a verified list of references. Every figure and table in the report is drawn from the project files and from the terminal output of the live runs.

The mandatory requirement that at least three orders are fulfilled and at least one is actively rejected with a clear customer reason is met in the offline run and in the final live run. Request 15 is rejected because 10,000 A4 sheets and 5,000 A3 colored sheets cannot arrive before the April 15 deadline (the earliest delivery is April 19) and cardboard is not stocked. The customer reply states each reason and confirms that no charge was made.

2. Problem Statement

The Beaver's Choice Paper Company sells paper, party supplies, and specialty products to schools, hotels, restaurants, city offices, and event organizers. Customers write in everyday language, and their messages are often emotional or urgent. A typical request asks for several products in different quantities and sets a delivery date for an event. Staff must read the message, find the matching catalog items, check stock, estimate supplier delivery, price the order, and record the sale.

Done by hand, this process is slow and error prone. Customer wording rarely matches catalog names, so staff must translate terms such as heavy cardstock or printer paper. Stock is limited, and the supplier needs between zero and seven days depending on the quantity ordered, so some requests cannot be met by the date the customer wants. Cash is finite, so restocking cannot be unlimited. Discount decisions are inconsistent unless they follow a rule that is informed by past quotes.

The problem is therefore to build an automated quoting and fulfilment workflow that is accurate, explainable, and safe with money. The workflow must handle impossible requests honestly, refuse what it cannot deliver, and never hide a dropped item behind a confirmation message.

The sample inquiries show how varied the demand is. One customer wants a few hundred sheets of cardstock for a ceremony. Another wants ten thousand sheets of paper plus party items for a festival. Some ask for products the company does not sell. The workflow must treat all these consistently, and it must explain its decision in language a customer can understand.

Requirements

- Use a recognized multi-agent framework with at most five agents, one of which orchestrates the others.
- Use all seven supplied helper functions through agent tools: create_transaction, get_all_inventory, get_stock_level, get_supplier_delivery_date, get_cash_balance, generate_financial_report, and search_quote_history.
- Evaluate the system on every request in the sample file and submit the results as a CSV file.
- Change the cash balance in at least three requests and fulfill at least three orders.

- Actively reject or leave unfulfilled at least one order, for example because of insufficient stock or an impossible supplier timeline, with a clear reason given to the customer.
- Give customers transparent answers that explain the decision, without exposing sensitive internal data.
- Document the architecture, the agent responsibilities, the results, and suggested improvements.

Constraints that make the problem hard

Three constraints drive the design. First, the catalog holds 46 products but only 18 are present in the starting inventory, so most requests involve items that must be ordered from the supplier. Second, the helper functions return only raw data, so judgement about discounts, reorder points, and spending limits must be built into the agents. Third, a language model can misread a message, so every number that reaches the database must come from deterministic code and not from free text.

A fourth, practical constraint became visible during live testing: a model may skip a step, repeat a step, or invent identifiers. The solution had to stay correct even when the model behaved badly.

3. Solution Statement

The solution is a hierarchy of five smolagents ToolCallingAgents. The orchestrator_agent receives the customer inquiry, registers the order, and delegates in a fixed order to the inventory_agent, quote_agent, finance_agent, and sales_agent. The orchestrator then returns a customer-facing reply. Each worker owns a small set of tools, and each tool wraps one or more of the supplied helper functions.

Four principles shaped the design. The first is that business rules live in plain functions, not in prompts, so arithmetic and database writes are exact and testable. The second is a shared order ledger: workers receive only an order key and read or update a typed order record, which removes transcription errors. The third is that the customer reply is always rendered from the ledger, never from model free text. The fourth is that every workflow step runs only from its own status, so a repeated call cannot corrupt an order.

Decision rules

The inventory agent maps customer wording to exact catalog names, compares stock with demand, and checks the supplier arrival date against the customer deadline. A line that cannot be supplied is rejected with a customer-safe reason, and the remaining lines proceed. When a sale would leave an item below its reorder point, the agent plans a top-up to twice that level.

Partial fulfilment is a deliberate choice. When one line of an order cannot be met, the system ships the lines that can be met and lists the rest with reasons, instead of refusing the whole order. This raises sales while keeping the customer informed. A single switch, ALLOW_PARTIAL_FULFILLMENT, changes the behaviour to all-or-nothing, in which case 10 of the 20 sample orders are fulfilled and 10 are rejected.

The quote agent prices accepted lines at catalog price and applies a volume discount tier. For orders that qualify, the discount is the average of the tier rate and the median discount found in matching historical quotes, capped at 15 percent. The finance agent approves purchases only if cash stays above a $1,000 reserve, and it drops optional safety purchases first when cash is tight. The sales agent writes the supplier purchases and customer sales to the database exactly once.

The solution also adds a reorder policy that the helper functions do not provide. Each item has a reorder point, taken from the inventory table or set to 100 units for new items. After a sale, any item below its reorder point is topped up to twice that level, and these purchases are the first to be dropped if cash is tight. This keeps popular items available without risking the cash reserve.

Table 2: Business parameters used by the agents

Parameter	Value	Purpose
Supplier cost ratio	60% of catalog price	Keeps discounted sales profitable
Cash reserve	$1,000	Never spent on replenishment
Default reorder point	100 units	For items not in the starting inventory
Safety top-up	2 times reorder point	Restores stock after a sale
Discount tiers	5% from 500, 7% from 1,000, 10% from 5,000 units	Rewards bulk orders
Discount cap	15%	Protects margin
Partial fulfilment	Enabled	Ships what can be supplied, explains the rest

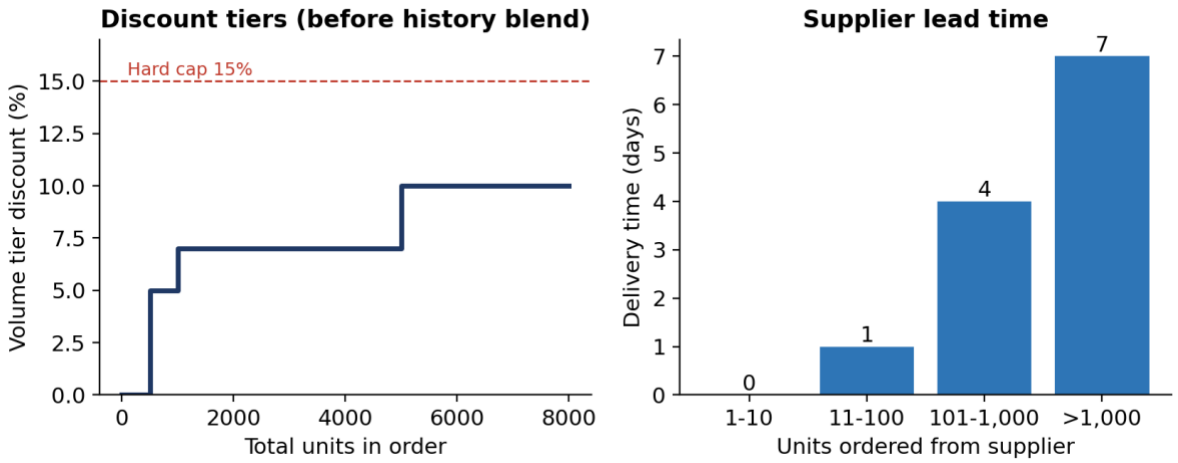


Figure 2: Discount tiers and supplier lead times that drive pricing and feasibility decisions.

The agents can be driven in two ways. By default, a deterministic RuleBasedModel emits real tool calls, so the evaluation is reproducible and runs offline. Setting USE_LLM=1 swaps in a live OpenAI-compatible model at temperature zero. Both modes use the same agents, tools, and safeguards.

4. Dataset

The project provides four data sources. Two are used to seed the database, one provides historical context for pricing, and one is the evaluation set. All files are comma-separated text files except the product catalog, which is defined inside the starter script.

Table 3: Data sources

Source	Records	Used for
Product catalog (paper_supplies)	46 items	Names, categories, unit prices
quotes.csv	100 historical quotes	Quote history for pricing context
quote_requests.csv	100 historical requests	Loaded into the quote_requests table
quote_requests_sample.csv	20 dated requests	Evaluation set (April 2025)
Seeded inventory table	18 items, 9,627 units, $4,940.30	Starting stock
Seeded transactions	18 stock orders and one cash seed	Starting cash of $45,059.70

The catalog contains 25 paper items, 15 product items such as cups and plates, 4 specialty items, and 2 large format items. Only 18 of the 46 items are in the starting inventory. During the evaluation 10 more items were stocked for the first time, which means that 18 catalog items were never needed.

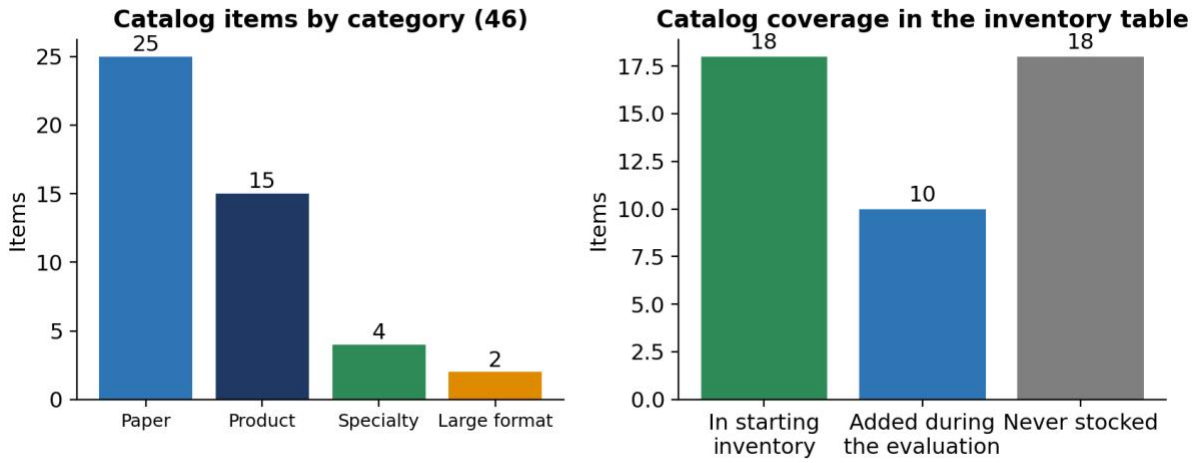


Figure 3: Catalog composition and its coverage in the inventory table.

The historical quotes include a total amount, a written explanation, and request metadata with the customer role, order size, and event type. Order sizes are balanced: 38 small, 27 medium, and 35 large. There are 12 customer roles and 15 event types. Quote totals are highly skewed. The median is $152, the mean is $6,910, and the maximum is $231,500, so a few very large orders dominate the average. One record has a total of negative one dollar, which is a data quality issue the system ignores because it reads only the discount wording in the explanations.

Two data issues deserve attention. The first is coverage: 28 of the 46 catalog items were stocked by the end of the evaluation, so the system depends on its ability to register new items in the inventory table. Without that step the financial report would ignore their stock and understate inventory value. The second is wording: the requests use terms such as printer paper, heavy cardstock, and poster board that do not appear in the catalog, so a mapping step is required.

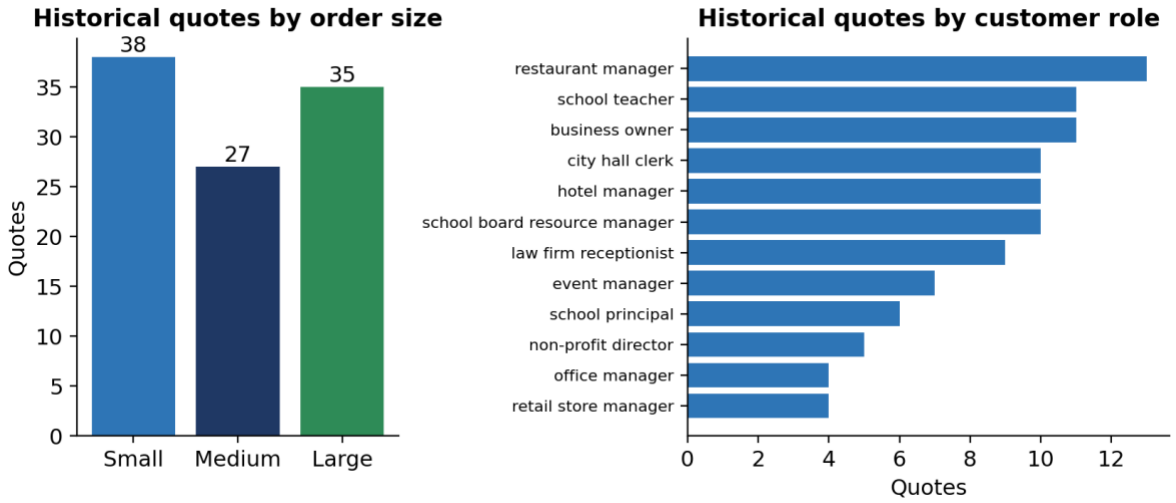


Figure 4: Historical quotes by order size and customer role.

The evaluation file has 20 requests dated between April 1 and April 17, 2025. Nine are small, five medium, and six large. The requests come from ten customer roles and twelve event types. Most name a delivery deadline, which makes them a good test of supplier lead time. They also include products that are not in the catalog, such as balloons, tickets, and cardboard, and sizes that the catalog does not offer, such as A3.

The sample requests were processed in date order, which matters because stock and cash carry over from one request to the next. Requests 12 and 13 share a date, so the sorted order processes request 13 first and the evaluation table lists them in that order. Because each request changes the database, the order of processing is part of the test and was kept identical in every run.

The historical explanations show a consistent pattern in which large orders receive about a 10 percent discount. The quote agent uses this pattern by extracting percentage discounts from explanations that match the requested items and event type. The first rows of the quotes file also show the style the company uses to explain a price, which the customer replies follow: items, unit prices, discount, and total.

5. Architecture

The architecture has four layers. The interface layer is the orchestrator agent, which accepts free text and returns a customer reply. The agent layer holds four workers, each with a focused tool set. The logic layer contains plain Python functions for catalog resolution, inventory assessment, quoting, authorization, and fulfilment. The data layer is the SQLite database accessed only through the seven starter helpers.

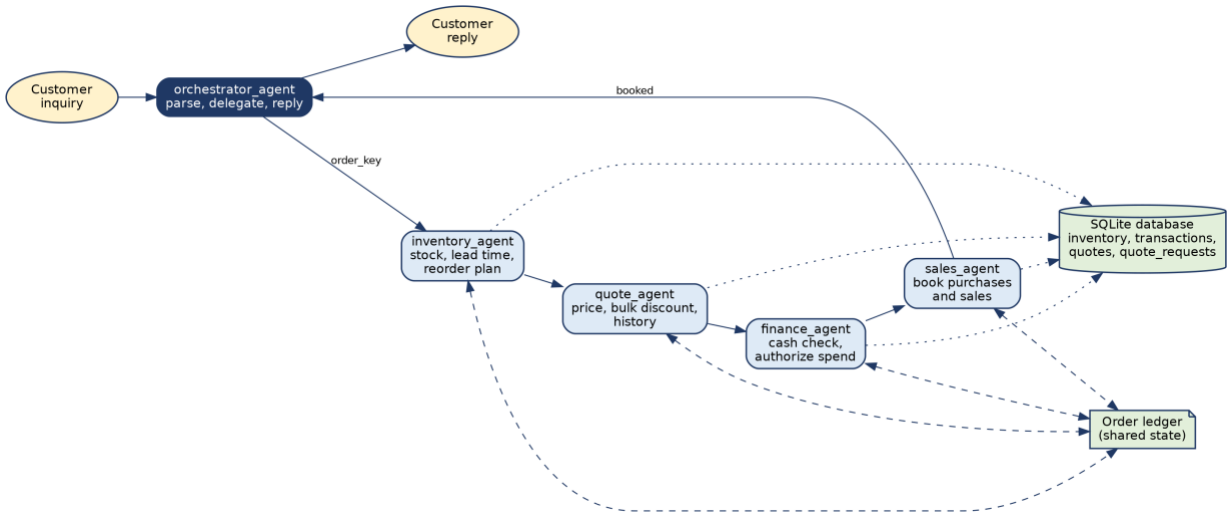


Figure 5: Five-agent architecture with the shared order ledger and database.

Control flows in one direction. The orchestrator registers the order and calls the inventory agent, which passes to the quote agent, then to the finance agent, then to the sales agent, and finally back to the orchestrator for the reply. Two exits exist. If no requested line can be supplied the flow ends after the inventory step, and if required purchases are unaffordable it ends after the finance step.

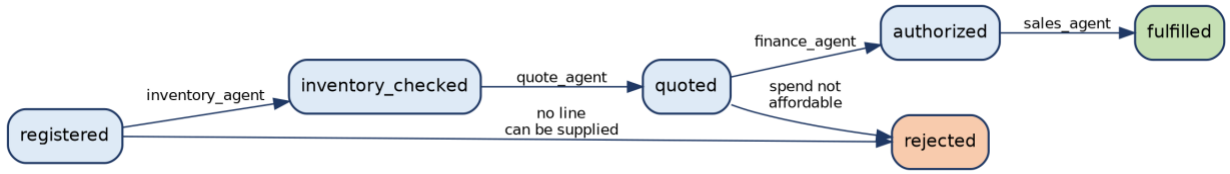


Figure 6: Order status transitions. Each worker acts only on the status that belongs to it.

The order ledger is a dictionary of typed OrderState records. Each record stores the request date, deadline, event, product lines, planned purchases, quote, status, and a booked flag. Because every step checks the status before it runs, repeated or out-of-order calls do nothing. This property was added after a live run showed a model re-running the inventory step on a finished order.

Table 4: Architecture layers

Layer	Components	Responsibility
Interface	orchestrator_agent	Parse inquiry, delegate, reply
Agents	inventory, quote, finance, sales	Call tools for their domain
Logic	assess_inventory, build_quote, authorize_purchases, fulfill_order	Exact business rules
State	ORDER_BOOK, ACTIVE_REQUEST	Share order data between agents
Data	Seven starter helpers and SQLite	Read and write all records
Models	RuleBasedModel or OpenAIServerModel	Choose the next tool call

Data flows through three channels. Tool calls carry the order key and a small number of simple arguments. The order ledger carries rich state, such as product lines and planned purchases. The database carries the permanent record of sales and purchases. Keeping these channels separate means that a mistake in a model's text cannot reach the permanent record.

The design follows the orchestrator-workers pattern described in published agent guidance, where a central model breaks a task down, delegates to workers, and combines their results. It also follows the advice to keep patterns simple and composable. The system uses fixed delegation order rather than open-ended planning, because the order workflow is well understood and predictability is more valuable than flexibility.

Safeguards sit around the model. The customer reply is rendered from the ledger. Any stage the agents leave unfinished is completed by the same functions the tools call. Tools ignore invented order keys, and they take the date, deadline, and event from the trusted request text. These measures make the outcome independent of how well a given model behaves, which is essential for a system that moves money.

Failure handling follows the same philosophy. If an agent run raises an error, the driver logs a warning and completes the remaining stages with the same functions the tools use. The order is never lost, and the customer always receives a reply that matches the ledger. This is why the third live run could match the offline run even though the live model took a different path through the tools.

6. Multi-Agents

The system has five agents and twelve tools. Table 5 lists them, and the subsections below describe each part.

Table 5: Agents and tools

Agent	Tools	Starter helper used
orchestrator_agent	register_order_tool, compose_customer_response_tool	None (order ledger)
inventory_agent	inventory_snapshot_tool, stock_check_tool, supplier_lead_time_tool, assess_order_inventory_tool	get_all_inventory, get_stock_level, get_supplier_delivery_date
quote_agent	quote_history_tool, build_quote_tool	search_quote_history
finance_agent	cash_balance_tool, financial_report_tool, authorize_purchases_tool	get_cash_balance, generate_financial_report
sales_agent	fulfill_order_tool	create_transaction

6.1 ToolCallingAgents

Every agent is a smolagents ToolCallingAgent, which emits structured tool calls instead of writing code. These suits validated business tools. The orchestrator receives the four workers as managed agents, each with a name and description. Workers run at most eight steps and the orchestrator twelve. The model is either the deterministic RuleBasedModel or, with USE_LLM=1, a gpt-4o-mini model at temperature zero.

Three design choices run through every agent. Tools return short strings or compact JSON, so the model has little room to misread them. Tools check status before acting, so repeated calls return a no-change message. Tools take dates and events from the trusted request text, because a live model sometimes copied them wrongly. Together these choices keep each agent small and keep the financial record exact.

6.2 inventory_agent

This agent maps customer wording to catalog names, compares stock with demand, and compares supplier arrival with the deadline. Lead time is up to 7 days for more than 1,000 units. It rejects lines that fail either test or plans a top-up to twice the reorder point when a sale leaves stock below it.

6.3 quote_agent

This agent searches historical quotes for the items and event, extracts past discount percentages, and prices each accepted line at catalog price. The discount is the average of the volume tier and the historical median, capped at 15 percent. Orders under 500 units receive none.

6.4 finance_agent

This agent reads the cash balance and a financial health report, then authorizes purchases. Purchases needed to fulfil the order must fit within cash minus a $1,000 reserve, otherwise the order is rejected. Optional safety purchases are dropped first when cash is tight.

6.5 sales_agent

This agent books the order. It registers any new item in the inventory table, writes supplier purchases, then writes one sale per line through create_transaction. A booked flag guarantees that an order is written once, and a check stops any sale that would push stock negative.

6.6 Orchestration tools

The orchestrator owns register_order_tool and compose_customer_response_tool. Registration stores the parsed items under the trusted key, date, deadline, and event. Composition renders the reply from the ledger, never from model text, and it lists items, prices, discount, delivery date, and a reason for every unsupplied line.

6.7 Helper functions

The seven starter helpers are the only route to data. Each is used by at least one tool, and a runtime counter proves it. Internal functions such as resolve_catalog_item and parse_request_text sit above them.

6.8 Database

The SQLite database has four tables: transactions, inventory, quotes, and quote_requests. The evaluation wrote 100 transactions, of which 42 were sales and 58 were supplier purchases, and added 10 inventory rows.

In the offline run each worker agent made between one and three tool calls per order. In the third live run the workers also made extra read-only calls, which is why the counts differ in Figure 13 while the number of database writes stays at 100.

Table 6: Starter helper functions and their agents

Helper	Used by tool	Agent
get_all_inventory	inventory_snapshot_tool, assess_order_inventory_tool	inventory_agent
get_stock_level	stock_check_tool, assess_order_inventory_tool	inventory_agent
get_supplier_delivery_date	supplier_lead_time_tool, assess_order_inventory_tool	inventory_agent
search_quote_history	quote_history_tool, build_quote_tool	quote_agent
get_cash_balance	cash_balance_tool, authorize_purchases_tool	finance_agent
generate_financial_report	financial_report_tool	finance_agent
create_transaction	fulfill_order_tool	sales_agent

Table 7: Database tables

Table	Content	Writer
transactions	Sales and stock orders with date, units, and price	sales_agent
inventory	Item, category, unit price, reorder point	Seed data and sales_agent
quotes	100 historical quotes with metadata	Seed data
quote_requests	100 historical requests	Seed data

7. Testing

Testing combined automated checks, fault injection, edge cases, and live model runs. The aim was to prove two things: that the business rules are right, and that the outcome does not depend on a model behaving well.

Built-in requirement checks

Every run ends by printing seven checks. They confirm that all 20 requests were evaluated, at least three were fulfilled, at least one was rejected with a stated reason, at least three changed cash, each of the seven helpers was called by a tool, each order was booked exactly once, and no reply contains cash, status, or error text. The helper check works by wrapping the helpers at run time and counting calls made from tools. The booking check compares booked sales with the sum of quoted totals.

The ledger reconciliation test recomputes cash from the transactions table and compares it with the balance reported by the agents. Sales minus purchases equals the reported cash exactly. The stock integrity test groups transactions by item and date and confirms that cumulative stock never falls below zero. Together they show that the books are internally consistent after every request.

Table 8: Test layers and results

Layer	What it proves	Result
Seven built-in checks	Activity requirements and booking integrity	7 of 7 pass
Ledger reconciliation	Cash equals sales minus purchases	Exact match
Stock integrity	No negative stock on any date	Pass
Faulty model simulation	Wrong key, broken JSON, skipped workers, leaky text	Identical output to clean run
Re-run model simulation	Repeated inventory call after booking	No duplicate booking
Cash-starved scenario	Unaffordable order is rejected safely	Rejected, reason shown
All-or-nothing mode	Policy switch works	10 fulfilled, 10 rejected
Determinism	Two offline runs agree	Identical replies
Live runs 1 to 3	Real model behaviour	Run 3 matches offline

Fault injection

A deliberately faulty model was written to copy failures seen in the first live run. It registered orders under a wrong key and the wrong date, sent broken JSON, called the sales agent early, and returned a free-text answer claiming a zero cash balance. With the safeguards in place, all 20 replies and the final cash balance were identical to the clean run. A second model repeated the inventory step after each order was booked. Before the fix this inflated sales to $8,238.34 against the correct $4,142.92. After the fix the figures matched.

Edge cases covered unaffordable orders, all-or-nothing mode, and parser behaviour. In the cash-starved scenario the database was left with only $500 of cash and an order for 9,000 sheets of cardstock was submitted. The finance agent rejected it, and the reply gave a customer-safe reason without revealing any financial detail. The parser was checked on lists that use commas, the word and, adjectives followed by commas, dates, sizes, and percentages, and it kept every item.

Live runs

The first live run reported only 5 of 20 orders as fulfilled, produced a zero cash balance in several replies, and leaked internal terms. The second run produced the same outcomes as the offline run but ended with cash of $47,095.30, which is $290.43 too high. Requests 8, 11, and 14 differed from the offline run by exactly that amount because a repeated inventory call had reset the order status. The third run, made after both defects were fixed, matched the offline run in every outcome, every cash change, and the final cash of $46,804.87.

Table 9: The three live runs

Run	Fulfilled (full or part)	Final cash	Finding
Live run 1	5 of 20	Not comparable	Replies contradicted the database, leaked internal terms
Live run 2	19 of 20	$47,095.30	Three orders booked twice, $290.43 overstated
Live run 3	19 of 20	$46,804.87	Matches offline run exactly

8. Results

The system fulfilled 19 of the 20 requests in full or in part and rejected one. Ten orders were complete, nine were partial, and request 15 was rejected. The requested product lines were supplied in 42 of 60 cases, which is 70 percent. The 18 declined lines had two causes: 14 could not be delivered by the customer deadline and 4 are not in the catalog (balloons, A3 paper, cardboard, and tickets).

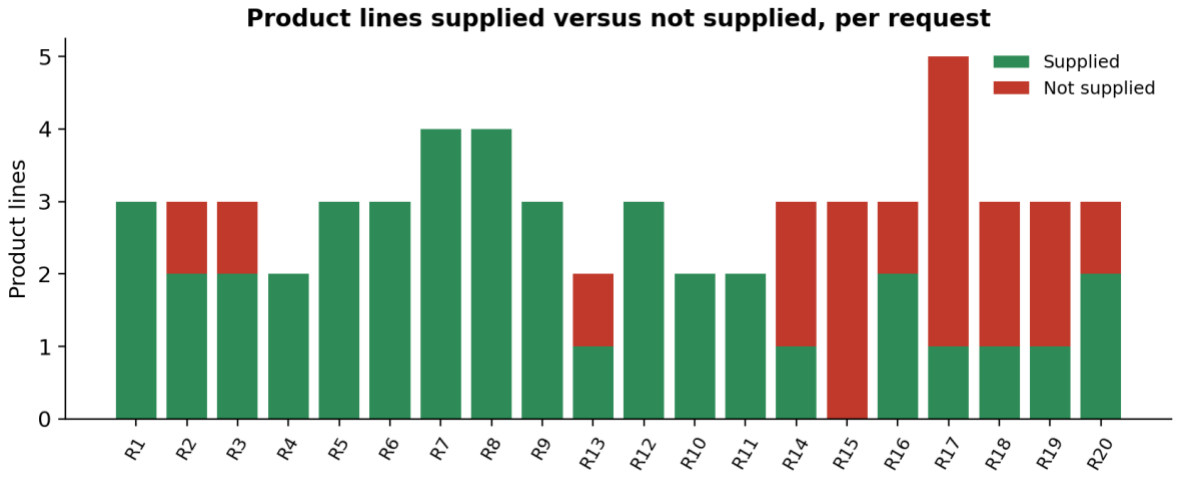


Figure 7: Product lines supplied and not supplied for each request.

Cash rose from $45,059.70 to $46,804.87, a gain of $1,745.17. The system booked $4,142.92 of sales and spent $2,397.75 on supplier purchases during the evaluation. Inventory value at list price fell from $4,940.30 to $4,404.55, so total assets rose from $50,000.00 to $51,209.42, a gain of 2.4 percent. The cash balance changed after 19 requests. The only request that did not change cash was the rejected one.

Quote values range from $20.00 to $1,125.00. The largest quote, for request 20, covers 7,000 units at a 10 percent discount. The smallest cash gains come from small orders that needed purchases, such as requests 18, 2, and 13, where purchases consumed most of the revenue. Request 17 changed cash by exactly its quote, because every unit it needed was already in stock and no purchase was required.

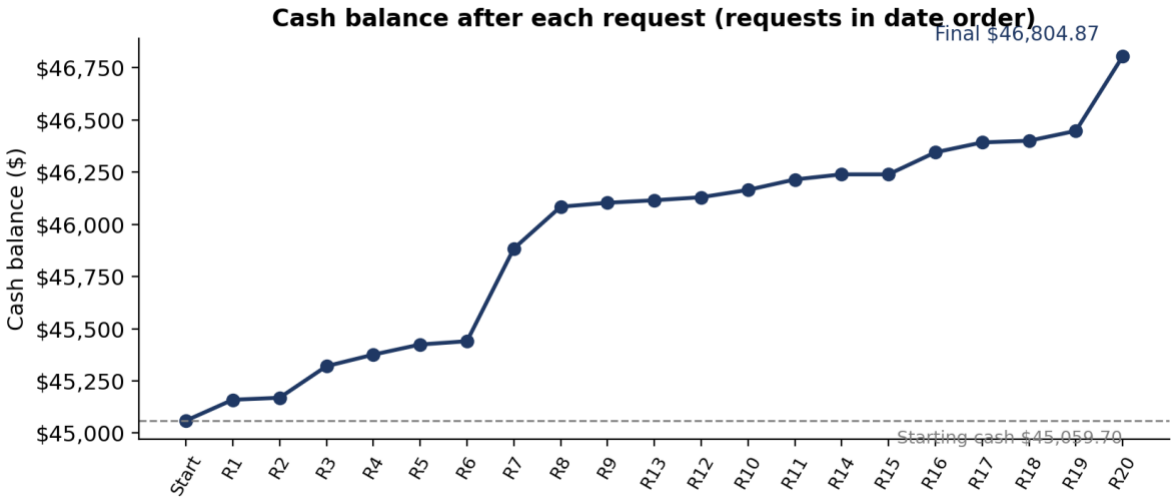


Figure 8: Cash balance after each request.

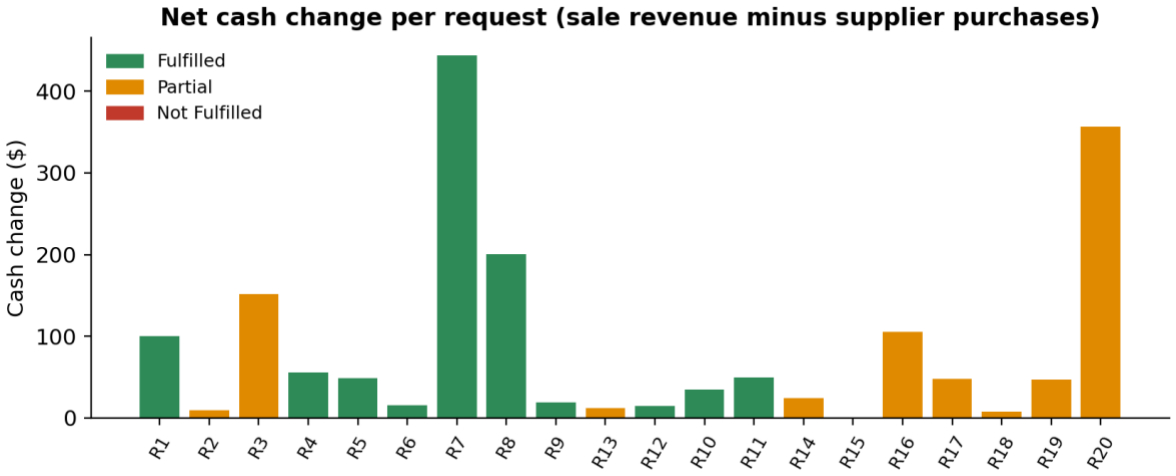


Figure 9: Net cash change for each request, colored by outcome.

Discounts followed the rules. Of the 19 booked orders, 5 received no discount, one received 5 percent, six received 7.5 percent, four received 8.5 percent, and three received 10 percent. Flyers earned the most revenue at $675.00, followed by poster paper at $565.62 and A4 paper at $471.56. A4 paper sold the most units, at 10,450.

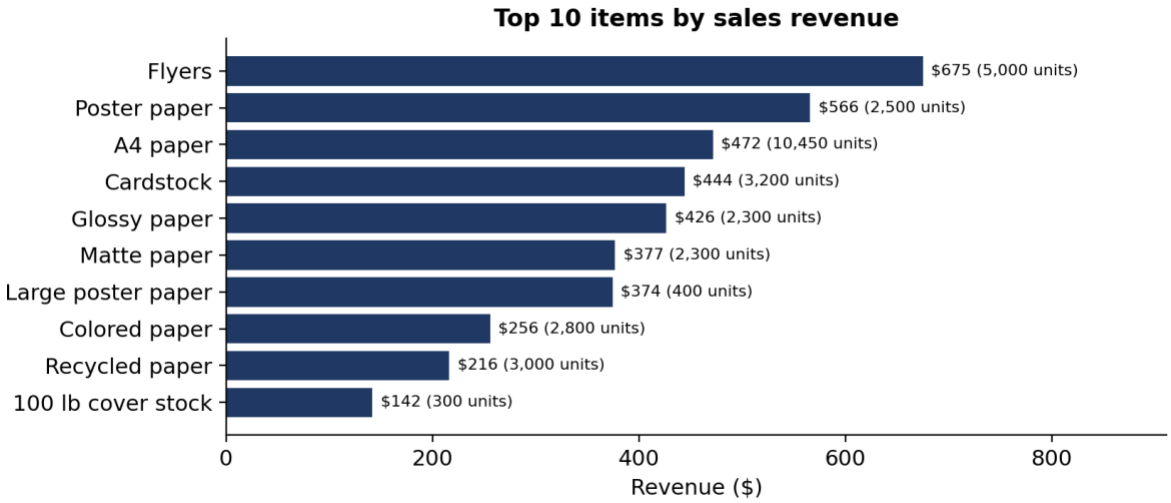


Figure 10: Top ten items by sales revenue.

Table 10 lists all 20 requests in the order they were processed. Each row combines the outcome, the quote, the cash effect, and the number of product lines supplied against those not supplied. Requests with a large share of lines not supplied, such as 14, 17, 18, and 19, are those whose deadlines were shorter than the supplier lead time for the quantities requested.

Table 10: Results for each request

Req.	Date	Outcome	Quote ($)	Cash change ($)	Lines ok / not
1	04-01	Fulfilled	100.00	100.00	3 / 0
2	04-03	Partial	129.50	9.50	2 / 1
3	04-04	Partial	468.00	151.26	2 / 1
4	04-05	Fulfilled	80.94	55.35	2 / 0
5	04-05	Fulfilled	123.53	48.53	3 / 0
6	04-06	Fulfilled	70.45	15.85	3 / 0
7	04-07	Fulfilled	622.20	443.76	4 / 0
8	04-07	Fulfilled	648.00	200.52	4 / 0
9	04-07	Fulfilled	32.50	19.00	3 / 0
13	04-08	Partial	30.00	12.00	1 / 1
12	04-08	Fulfilled	48.10	14.50	3 / 0
10	04-08	Fulfilled	134.12	35.12	2 / 0

Req.	Date	Outcome	Quote ($)	Cash change ($)	Lines ok / not
11	04-08	Fulfilled	142.45	50.05	2 / 0
14	04-09	Partial	69.38	24.38	1 / 2
15	04-12	Not Fulfilled	0.00	0.00	0 / 3
16	04-13	Partial	114.00	105.60	2 / 1
17	04-14	Partial	47.50	47.50	1 / 4
18	04-14	Partial	20.00	8.00	1 / 2
19	04-15	Partial	137.25	47.25	1 / 2
20	04-17	Partial	1,125.00	357.00	2 / 1

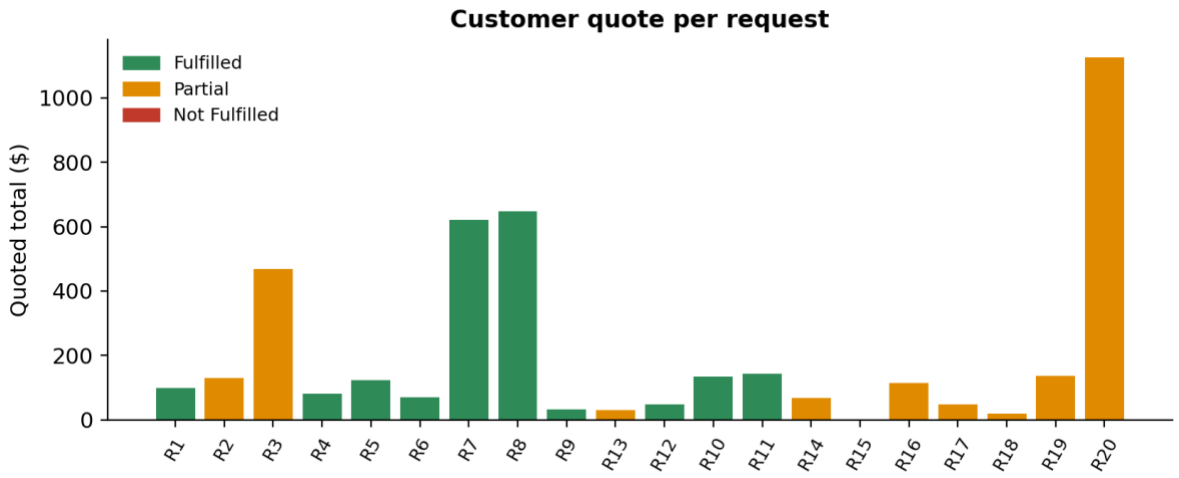


Figure 11: Customer quote for each request.

Request 15 is the required rejection. The customer asked for 10,000 A4 sheets and 5,000 A3 colored sheets by April 15. Each large quantity needs seven days from the supplier, which means April 19, and the reply says so. It also states that cardboard is not in the catalog and confirms that no charge was made. Requests 14, 19, and several others were shipped in part for the same deadline reason.

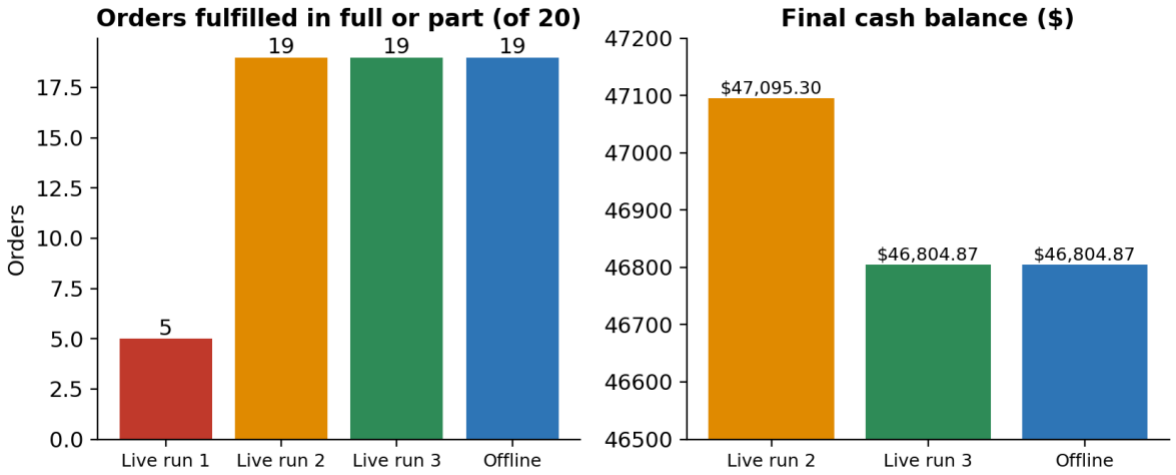


Figure 12: Progress across the live runs compared with the offline run.

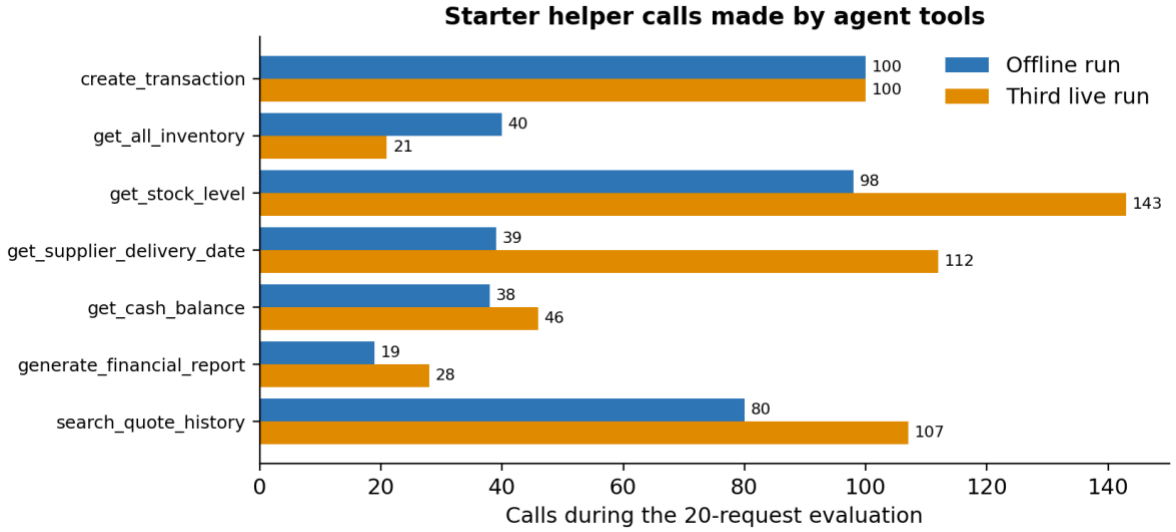


Figure 13: Helper calls made by tools in the offline run and in the third live run.

Both runs made exactly 100 create_transaction calls, which shows that no order was written twice. The live model made more read-only calls, for example 112 supplier date lookups against 39 offline, because it explores tools. These calls change no data.

9. Value Statement

The system gives the company a consistent, explainable quote desk. Each of the 20 requests received a structured reply within one run, with itemized prices, a stated discount and its reason, an estimated delivery date, and a reason for every unsupplied line. Customers are told early what cannot be done, and no order is confirmed when it cannot be delivered.

Financial value comes from controlled decisions. Supplier cost is modelled at 60 percent of catalog price, discounts are capped at 15 percent, and a $1,000 cash reserve is never spent. During the evaluation the business gained $1,745.17 in cash and $1,209.42 in total assets while serving 19 of 20 requests. Safety purchases kept stock available for later orders, and 33 of the 58 purchases were of this kind.

Table 11: Value delivered

Area	Evidence	Benefit
Accuracy	No dropped items, 42 of 60 lines supplied, 18 declined with reasons	Customers are never misled
Consistency	Rules, not opinions, set discounts and feasibility	Same answer every time
Cash protection	$1,000 reserve, affordability check before purchases	Lower risk of overspending
Margin	60 percent cost model, 15 percent cap	Profitable discounts
Inventory	Reorder top-up, 10 items newly stocked	Fewer stock-outs
Auditability	Every sale and purchase in the transactions table	Easy review
Speed	20 requests handled in one run	Staff time released

Operational value comes from transparency. The ledger holds every decision, and booked sales reconcile exactly with quoted totals. A reviewer can trace any number in a reply to a database row. The seven built-in checks give a quick signal after every run, and the booking check would flag a repeat of the duplicate booking defect immediately.

The pricing logic gives a clear commercial benefit. Five of the 19 booked orders received no discount, which preserves margin where volume does not justify one. The larger orders received between 5 and 10 percent, in line with the 10 percent that

historical quotes show for large orders. This makes pricing predictable for customers and defensible for staff.

The safeguards also lower operational risk. A reviewer can see exactly why an order was shipped in part, why a line was declined, and why a discount was applied. If a customer disputes a quote, the ledger and the transactions table provide the evidence. This traceability is difficult to achieve with a purely conversational agent whose decisions exist only in generated text.

Strategic value lies in the pattern. The same architecture can take on new products, new suppliers, or a different pricing policy by changing tools and parameters. Because business rules sit outside the model, the company can change the language model without changing its financial behaviour. The live runs showed that this separation matters: the model made mistakes, yet the customer replies and the books stayed correct once the safeguards were in place.

Finally, the work produced a reusable testing method. Fault injection, re-run simulation, and ledger reconciliation found defects that ordinary runs hid. These techniques apply to any agent system that writes to a financial record.

10. Limitations

Nine of the 20 requests were fulfilled only in part. Fourteen of the 18 declined lines failed on supplier lead time, because orders above 1,000 units need seven days and many customers asked for delivery within five to ten days. A customer may therefore receive a fraction of what was requested. The system explains this clearly, but it does not offer split deliveries or substitutes.

Catalog matching is rule based. Words such as recycled, biodegradable, and A5 are not catalog attributes, so the system ignores them or adds a note. A3 glossy and matte paper are matched to the size-agnostic catalog item with a visible note, while generic A3 paper is declined. This is a judgement call that a business owner may want to change.

Table 12: Limitations and mitigations

Limitation	Effect	Possible mitigation
Seven-day lead time above 1,000 units	14 lines declined	Split delivery or earlier reorders
Rule-based catalog matching	Qualifiers ignored	Attribute-aware catalog
Cost modelled at 60 percent of price	Margin is an estimate	Real supplier cost data
Booking on request date	No delayed stock arrival	Model arrival as an inventory event
Fixed discount tiers	Not margin aware	Learned pricing model
Live model variability	Extra tool calls, possible mis-parsing	Safeguards, stricter schemas
Single evaluation set	20 requests only	Test on the 100 historical requests

Several assumptions simplify reality. Supplier cost is assumed to be 60 percent of catalog price because the data has no cost field. Sales and the matching purchases are booked on the request date, and supplier arrival is checked against the deadline but not modelled as a later inventory event. Units such as reams and packets are treated one to one with catalog units, which matches how the historical quotes read.

The evidence for live model behaviour is limited. The model was run three times, and the first two runs found defects, so there may be further failure patterns that these runs

did not reveal. The safeguards make the outcome independent of the model for the steps that touch money, but a live model can still transcribe an item badly. In that case an order may be declined with an accurate reason that is nevertheless wrong for the customer.

Scale is another limit. The evaluation used 20 requests and a small database, so run time, concurrency, and cost per request were not measured. The ledger is an in-memory dictionary that is cleared between runs, which suits a batch evaluation but not a long-lived service with many simultaneous customers. A production version would need a persistent order store and locking.

Extra tool calls are a further cost, because the live model sometimes calls tools that are not needed, such as repeated supplier date lookups. These calls are harmless to the data, but they add cost and delay, and a production system would need to limit them with tighter instructions or a cap on steps.

The terminal output of the final live run starts at request 1 and does not print the line that names the model, so the live mode is taken from the run description. Finally, the historical quotes contain outliers and one negative total, and the discount extraction relies on the wording of explanations. A cleaner history would give the quote agent a stronger basis.

11. Conclusion

The project set out to build a multi-agent system that can quote, check stock, protect cash, and record sales for a paper company, while rejecting what it cannot deliver. The final system does this. It uses five smolagents agents, twelve tools, and all seven starter helpers. On the 20 sample requests it fulfilled 10 orders in full, 9 in part, and rejected 1 with a clear reason.

The mandatory requirement is met. At least three orders were fulfilled, and request 15 was actively rejected with a clear customer reason: the supplier cannot deliver the large quantities by the deadline, the earliest date is April 19, and cardboard is not stocked. The cash balance changed in 19 requests, every helper was used, and all seven built-in checks pass in both the offline run and the third live run.

The most important lesson came from the live runs. A design that looks correct under a deterministic model can fail under a real one. The first run showed that model text cannot be trusted for customer replies or identifiers. The second showed that a repeatable step can corrupt a ledger if its status is not protected. The fixes were small but decisive: render replies from the ledger, complete unfinished stages in code, ignore invented keys, and guard every step by status and a booked flag. Reproducing each defect under a simulated model made the fixes testable.

The architecture also shows how to use agents wisely. The model chooses which tool to call, while code decides every price, quantity, and database write. This division keeps the benefits of agent coordination and removes its main financial risk. The orchestrator-workers pattern fits a workflow whose steps are known and ordered.

The report also documents what was learned from review. An earlier version of the solution crashed under the real framework, dropped items without telling the customer, and produced results through a silent fallback. Replacing it with the current design and proving the design with simulated failures and live runs, is the main result of the work.

Future work should focus on the largest source of partial fulfilment, which is supplier lead time. Split deliveries, substitute products, demand-driven reorder points, a clarification loop for ambiguous items, and a margin-aware pricing model would turn many of the nine partial orders into complete sales. The final live run matched the offline run exactly, so these extensions can be added with confidence that the base system is stable.

In summary, the system is accurate, explainable, and safe with money, and its behaviour is reproducible across a deterministic model and a live model.

12. References

Each reference below was checked on October 7, 2026, by searching for it and opening or reading the returned page. All eight were found. The table records the result, and the notes explain how each source informed the project. Software used in the project (Python, pandas, SQLAlchemy, smolagents, matplotlib, Graphviz, and Mermaid) is cited only through the documentation pages listed here.

Table 13: Reference verification

No.	Reference
1	Hugging Face. smolagents documentation: Orchestrate a multi-agent system. https://huggingface.co/docs/smolagents/examples/multiagents
2	Hugging Face. smolagents reference: MultiStepAgent and agents. https://huggingface.co/docs/smolagents/reference/agents
3	Anthropic. Building effective agents (December 2024). https://www.anthropic.com/research/building-effective-agents
4	Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. https://arxiv.org/abs/2210.03629
5	Wu, Q. et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. https://arxiv.org/abs/2308.08155
6	SQLAlchemy. SQLAlchemy 2.0 Documentation, Unified Tutorial. https://docs.sqlalchemy.org/en/20/tutorial/index.html
7	pandas. pandas documentation and user guide. https://pandas.pydata.org/docs/
8	Mermaid. Flowcharts, basic syntax. https://mermaid.js.org/syntax/flowchart.html